

MODULE 5.1 · Concept 3 of 7

Tool Use and Function Calling

How modern LLMs support structured tool calls natively through instruction tuning

9 min

Duration

B3 – Apply

Bloom's

L2 – Intermediate

Level

TH-000002

Prerequisite

AI-AG-IN-TH-000003

Prof. Sashikumaar Ganesan · IISc Bangalore · AhamX Bodhi

SECTION 1 — WHAT

Tool Use for Agents

Extending LLM capabilities beyond text generation into real-world actions

Will Start
@ 11.05

What Tool Use Means for Agents

Without tools, agents can only generate text. With tools, they can act.

Without Tools (Text Only)

- Can answer questions from training data
- Cannot access current information
- Cannot execute code or query databases
- Cannot book, purchase, or modify anything
- Output: text strings only

With Tools (Full Agent)

- Search the web for current information
- Query databases and APIs
- Execute Python code in sandboxes
- Book flights, send emails, modify files
- Output: text + real-world actions

Tools transform an LLM from a text generator into an agent that can affect the real world.

The Function Calling Pipeline

Define → Select → Execute → Inject — four steps per tool call

1

Define Tools

Developer provides tool schemas: name, description, parameters as JSON Schema. Added to system prompt. The agent 'knows' what tools exist.

2

LLM Selects

Based on the Thought reasoning, the LLM chooses which tool to call and fills in the parameters. Generates a structured JSON object, not free text.

3

Orchestrator Executes

Your code parses the tool call, validates parameters, executes the real API/DB/code operation, and captures the result string.

4

Result Injected

Tool result is added to context as an Observation. LLM reasons about it and decides: call another tool, or generate the final answer.

Defining Tool Schemas: JSON Schema Format

Name + description + parameters — the contract between agent and tool

```
{
  "name": "web_search",
  "description": "Search the web for current information. Use for any query about current events, prices, people, or facts that may have changed since training. Do NOT use for calculations.",
  "parameters": {
    "type": "object",
    "properties": {
      "query": {
        "type": "string",
        "description": "The search query. Be specific and include dates if relevant."
      },
      "max_results": {
        "type": "integer",
        "description": "Number of results to return. Default: 5.",
        "default": 5
      }
    },
    "required": ["query"]
  }
}
```

if possible (the same)
User → "sum/2cm to improve to tool description"

Does

SKILLING

TI +
TR
Name: Deriv

web search() (is path)

Model Selecting and Populating Tool Calls

How the LLM decides which tool to use and fills in parameters

Given these 3 tools in the system prompt:

web_search

Search web for current information

calculator

Evaluate mathematical expressions

code_executor

Run Python code in sandbox

How the LLM decides — matching user intent to tool descriptions:

User Query	Tool Selected	Why This Tool
'What is AAPL stock price?'	web_search	Needs current data → matches 'current information' in description
'What is 245×18.7 ?'	calculator	Math expression → matches 'mathematical expressions' description
'Generate first 20 primes'	code_executor	Needs code execution → matches 'Run Python code' description

Orchestrator Executes: Your Code Is Always the Intermediary

The model never touches the real API — you validate and execute

Orchestrator pseudocode

```
def execute_tool_call(tool_call):  
    name = tool_call["tool"]  
    args = tool_call["args"]  
  
    # 1. Validate tool exists  
    if name not in tool_registry:  
        return "Error: Unknown tool '{name}'"  
  
    # 2. Validate parameters  
    schema = tool_registry[name].schema  
    if not validate(args, schema):  
        return "Error: Invalid parameters"  
  
    # 3. Execute with timeout  
    try:  
        result = tool_registry[name].execute(**args, timeout=30)  
    except TimeoutError:  
        return "Error: Tool timed out after 30s"  
    except Exception as e:  
        return f"Error: {e}"  
  
    # 4. Return result (injected as Observation)  
    return str(result)
```



Real Tool Schemas: Four Production Examples

Web search, calculator, code execution, database query

Tool	Parameters	Returns
web_search(query)	query: string max_results: int (default 5)	JSON array of {title, url, snippet}
calculator(expr)	expression: string (e.g., "245 * 18.7")	Numerical result as string
code_exec(code, lang)	code: string (Python) lang: "python" (default)	stdout + stderr + return_code
db_query(sql)	sql: string (SELECT only) database: string	JSON array of row objects

These four tools cover ~80% of production agent use cases. Most agents need only 2–3 tools.

SECTION 2 — HOW

Registries, Validation, and Error Handling

Making tool calling robust in production

Tool Registries: Static vs. Dynamic Discovery

How agents know what tools are available

Static Registry

- Tools defined at agent initialization
- Hardcoded in the system prompt
- Cannot change during execution
- Simpler, more predictable
- Used by: most LangChain agents

Best when: tool set is fixed and known.

Dynamic Registry (MCP)

- Tools discovered at runtime via protocol
- MCP servers expose tool schemas on request
- Agent can connect to new tools mid-session
- More flexible, but more complex
- Used by: Claude, MCP-compatible agents

Best when: tool set varies by user or task.

Hybrid

A2A - Google

Multi-Tool Selection: Description Quality Matters

When an agent has 10+ tools, the description IS the selection mechanism

Bad descriptions → wrong tool selection:

✗ Bad: "search" – "Search things"

✓ Good: "web_search" — "Search the web for current information about events, prices, people, or facts. Do NOT use for calculations or code execution."

✗ Bad: "calc" – "Do math"

✓ Good: "calculator" — "Evaluate a mathematical expression. Input must be a valid math expression string like '245 * 18.7'. Returns numerical result."

Include negative examples ('Do NOT use for X') to prevent common misroutes.

Tool Call Validation and Error Handling

What to check before executing — and what to do when tools fail

Failure Type	What Happens	Your Response
Unknown tool name	Model hallucinates a tool that doesn't exist	Return: "Error: tool 'X' not found. Available: [list]"
Invalid parameters	Wrong type, missing required, out of bounds	Return: "Error: param 'date' must be YYYY-MM-DD format"
Timeout	API takes >30 seconds to respond	Return: "Error: timed out". Agent can retry or use fallback.
Rate limit	Too many API calls in short window	Queue and retry with backoff. Inject wait message.
API error	500 server error, auth failure, etc.	Return: "Error: API returned 500". Agent reformulates.

⋮

Always return errors as Observations — let the agent reason about them and retry. Never crash silently.

Tool Calling Across LLM Providers

Same concept, different formats — all instruction-tuned, no fine-tuning needed

Provider	Format	Key Feature	Since
OpenAI GPT-4	function_call + tool_choice	Parallel tool calls, forced tool selection	June 2023
Anthropic Claude	tool_use content blocks	Thinking blocks visible alongside tool use	March 2024
Meta Llama 3.1+	< tool_call > special tokens	Open-source, self-hostable	July 2024
Mistral / Mixtral	[TOOL_CALLS] format	Efficient, good code task support	Feb 2024
Google Gemini	function_declarations	2M context for complex multi-tool tasks	Feb 2024

Key takeaway: Tool calling is instruction-tuned, NOT fine-tuned. All major LLMs support it out of the box. You just define the schema.

Key Takeaways

The essential points from this concept

- Tools transform LLMs from text generators to agents** that can act in the real world.
- Four-step pipeline:** Define schemas → LLM selects → Orchestrator executes → Result injected.
- Description quality determines selection accuracy** — include negative examples and be specific.
- Always validate before execution:** check tool name, parameter types, bounds, and permissions.
- All major LLMs support tool calling natively** — GPT-4, Claude, Llama 3.1+, Mistral, Gemini.

Next: AI-AG-IN-VZ-000001 — ReAct Loop Visualization

AI-AG-IN-TH-000003 Complete

Tool Use and Function Calling

Concept 3 of 8

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi